



Πανεπιστήμιο Πειραιώς
Σχολή Τεχνολογιών Πληροφορικής και Τηλεπικοινωνιών
Τμήμα Ψηφιακών Συστημάτων

Επίπεδο: Μεταπτυχιακό Πρόγραμμα Σπουδών

Προχωρημένα Θέματα Κυβερνοασφάλειας και Τεχνητής Νοημοσύνης

LLM-Enhanced Honeypot and Attacker Interaction Analytics

Επιβλέποντες Καθηγητές: Χρήστος Ξενάκης, Αποστόλης Ζάρρας και
Γεώργιος Βούρος

Καλαϊτζίδης Ιωάννης

ioannis_kalaitzidis@ssl-unipi.gr

Πειραιάς

12/07/2026

Εισαγωγή

Τα honeypots αποτελούν μία από τις πιο διδακτικές τεχνικές άμυνας στην κυβερνοασφάλεια. Αντί να προσπαθούν να αποκλείσουν τον επιτιθέμενο, τον προσκαλούν σε ένα ελεγχόμενο, ψεύτικο περιβάλλον που δεν έχει καμία πραγματική επιχειρησιακή αξία. Επειδή κανένας νόμιμος χρήστης δεν έχει λόγο να αγγίξει ένα τέτοιο σύστημα, κάθε αλληλεπίδραση μαζί του είναι σχεδόν εξ ορισμού ύποπτη, οπότε παράγει καθαρή πληροφορία για τις τακτικές και τις προθέσεις του αντιπάλου, χωρίς τον θόρυβο της νόμιμης κίνησης (διάλεξη 1). Η αξία ενός honeypot όμως εξαρτάται απόλυτα από την πειστικότητά του. Ένα περιβάλλον που προδίδει άμεσα τον εαυτό του δεν συλλέγει τίποτα, καθώς ο επιτιθέμενος αποχωρεί μόλις αντιληφθεί την παγίδα.

Αντικείμενο της παρούσας εργασίας είναι η κατασκευή ενός honeypot SSH εμπλουτισμένου με περιεχόμενο παραχθέν από Μεγάλα Γλωσσικά Μοντέλα (LLM), και η συστηματική ανάλυση της συμπεριφοράς των επιτιθεμένων που αλληλεπιδρούν με αυτό. Το ερώτημα δεν είναι απλώς αν ένα honeypot μπορεί να καταγράψει επιθέσεις, αλλά πόσο πειστικά δολώματα μπορούμε να παράγουμε, πόσο βαθιά εμπλοκή προκαλούν, και τι μπορούμε να συμπεράνουμε για τις προθέσεις των επιτιθεμένων μέσα από μη επιβλεπόμενη μάθηση (διάλεξη 3).

Καθοριστική σχεδιαστική αρχή όλης της εργασίας είναι ότι η ανάλυση πρέπει να βασίζεται σε πραγματικά γεγονότα και όχι σε εικασίες. Για τον λόγο αυτό, το honeypot που κατασκευάσαμε δεν περιορίζεται σε μια απλή μίμηση, αλλά προσομοιώνει ένα πλήρες διαδραστικό κέλυφος και σερβίρει πραγματικά τα δολώματα. Έτσι, κάθε μετρική (για παράδειγμα, ποια αρχεία άνοιξε ο επιτιθέμενος) προκύπτει από πραγματικά καταγεγραμμένα συμβάντα. Η προσέγγιση αυτή συνδέεται με τις αρχές της υπεύθυνης τεχνητής νοημοσύνης, και ειδικότερα με την επεξηγησιμότητα και τη λογοδοσία (διάλεξη 7).

Η εργασία απαντά σε επτά ζητούμενα. Θα σχεδιαστεί το honeypot και το μοντέλο απειλής (Task 1), θα παραχθούν τα decoy artefacts με έμφαση στα LLM-generated (Task 2), θα υλοποιηθεί δομημένη καταγραφή συνεδριών (Task 3), θα εκτελεστούν ελεγχόμενα playbooks επιτιθεμένου (Task 4), θα αναλυθεί η συμπεριφορά με clustering και anomaly detection (Task 5), θα αξιολογηθεί η ποιότητα της παραπλάνησης και ο κίνδυνος fingerprinting (Task 6), και θα εξεταστούν τα ζητήματα ηθικής, ασφάλειας και επιχειρησιακής χρήσης (Task 7). Υλοποιήθηκε επιπλέον ένα προαιρετικό προχωρημένο task supervised ταξινόμησης πρόθεσης.

Δομή της υλοποίησης

Το σύστημα δεν είναι ένα ενιαίο πρόγραμμα, αλλά μια αλυσίδα διακριτών scripts σε Python, το καθένα με μία σαφή αρμοδιότητα. Το **00_config.py** ορίζει ένα κοινό system profile (ονόματα, εκδόσεις, διευθύνσεις, ημερομηνίες) που τροφοδοτεί όλα τα υπόλοιπα, εξασφαλίζοντας εσωτερική συνέπεια. Το **01_generate_decoys.py** παράγει τα δολώματα. Το **02_ssh_honeypot.py** υλοποιεί το honeypot και την καταγραφή. Το **03_run_playbooks.py** εκτελεί τα σενάρια επίθεσης. Το **04_analyze_sessions.py** κάνει την ανάλυση συμπεριφοράς, το clustering και τα γραφήματα. Το **05_intent_classifier.py** υλοποιεί τον προαιρετικό ταξινομητή. Ένας ενιαίος orchestrator (**run_all.py**) τρέχει όλη την αλυσίδα, τόσο σε Linux/Kali όσο και σε Windows.

Όλη η ροή είναι επαναλήψιμη μέσω ενός σταθερού random seed (τιμή 25012), οπότε κάθε αριθμός της αναφοράς προκύπτει από την ίδια, ενιαία εκτέλεση. Το honeypot ακούει αποκλειστικά στο loopback (127.0.0.1) και όλα τα δεδομένα είναι πλασματικά.

Task 1: Σχεδίαση και ανάπτυξη του honeypot

Η εκφώνηση δεν ορίζει ένα συγκεκριμένο σενάριο, αλλά ζητά ένα πειστικό περιβάλλον με συνεπές υπόβαθρο. Η επιλογή του σεναρίου έγινε από εμάς. Επιλέχθηκε ένας τυπικός εσωτερικός εξυπηρετητής εφαρμογής Linux (application server), με όνομα app-srv-01, λειτουργικό Ubuntu 22.04.4 LTS και OpenSSH 8.9p1, γιατί αποτελεί έναν από τους πιο συνηθισμένους πραγματικούς στόχους. Οι εξυπηρετητές αυτού του τύπου φιλοξενούν εφαρμογές web, συνδέονται με βάσεις δεδομένων και βρίσκονται σε εσωτερικά εταιρικά δίκτυα, οπότε ένας επιτιθέμενος που αποκτά πρόσβαση σε έναν τέτοιο εξυπηρετητή έχει κάθε λόγο να τον εξερευνήσει.

Σε αντίθεση με μια απλή μίμηση banner, το honeypot υλοποιεί πραγματικό πρωτόκολλο SSH μέσω της βιβλιοθήκης paramiko, δηλαδή κανονική ανταλλαγή κλειδιών, authentication και διαδραστικό κανάλι. Ένας επιτιθέμενος συνδέεται με έναν κανονικό ssh client (για παράδειγμα `ssh -p 2222 root@127.0.0.1`) και λαμβάνει ένα προσομοιωμένο κέλυφος, όπως θα συνέβαινε σε ένα πραγματικό σύστημα.

Μοντέλο απειλής

Το honeypot είναι σχεδιασμένο για να μελετά έναν επιτιθέμενο ο οποίος έχει ήδη αποκτήσει πρόσβαση σε ένα κέλυφος (στάδιο post-exploitation) και εξερευνά το σύστημα. Οι στόχοι που υποθέτουμε είναι πέντε: η αναγνώριση του περιβάλλοντος, η αναζήτηση credentials, η αναβάθμιση προνομίων, ο εντοπισμός δεδομένων προς εξαγωγή και η προετοιμασία οριζόντιας μετακίνησης.

Ο όρος οριζόντια μετακίνηση (lateral movement) περιγράφει την προσπάθεια του επιτιθέμενου να μετακινηθεί από το μηχάνημα στο οποίο βρίσκεται προς άλλα μηχανήματα του ίδιου εσωτερικού δικτύου. Για παράδειγμα, αφού αποκτήσει πρόσβαση στον εξυπηρετητή εφαρμογής, ο επιτιθέμενος συχνά επιχειρεί να συνδεθεί στη βάση δεδομένων ή σε άλλους εσωτερικούς εξυπηρετητές, ώστε να επεκτείνει την παρουσία του. Το honeypot δεν προσπαθεί να αποκλείσει καμία από αυτές τις ενέργειες, αλλά να τις παρατηρήσει και να τις καταγράψει.

Αρχιτεκτονική

Η αρχιτεκτονική οργανώνεται σε τέσσερα στρώματα, καθένα από τα οποία έχει διακριτό ρόλο:

1. **Το δικτυακό στρώμα (paramiko Transport):** Δέχεται την εισερχόμενη σύνδεση TCP, ξεκινά τον διάλογο SSH ως εξυπηρετητής και παρουσιάζει το banner OpenSSH 8.9p1. Ταυτόχρονα καταγράφει το banner του πελάτη (client_version), το οποίο αποκαλύπτει το εργαλείο που χρησιμοποιεί ο επιτιθέμενος και αποτελεί ένα πρώτο στοιχείο fingerprinting του ίδιου του αντιπάλου.
2. **Το στρώμα authentication:** Το honeypot δέχεται τις προσπάθειες σύνδεσης (τυπική honeypot συμπεριφορά, ώστε ο επιτιθέμενος να προχωρήσει και να παρατηρηθεί), αλλά καταγράφει κάθε προσπάθεια, μαζί με το όνομα χρήστη και τη μέθοδο. Για λόγους ασφαλείας καταγράφεται μόνο ένα δείγμα του κωδικού, ποτέ ολόκληρος.
3. **Ο προσομοιωτής κελύφους (shell emulator):** Αυτό είναι το πιο σύνθετο στρώμα. Λαμβάνει κάθε γραμμή εντολής, την αναλύει (parsing), αναγνωρίζει την εντολή και τα ορίσματά της, και την εκτελεί πάνω στο εικονικό σύστημα αρχείων. Υποστηρίζει ένα ρεαλιστικό υποσύνολο εντολών (ls, cd, cat, grep, find, ps, ss, id, uname, sudo, history και άλλες), χειρίζεται σωστά διαδρομές (απόλυτες, σχετικές, με ~), και κρατά κατάσταση ανά συνεδρία, δηλαδή θυμάται τον τρέχοντα κατάλογο ώστε οι διαδοχικές εντολές cd και ls να συμπεριφέρονται όπως σε πραγματικό κέλυφος. Το πιο σημαντικό είναι ότι, όταν ο επιτιθέμενος διαβάσει ένα δόλωμα, ο προσομοιωτής επιστρέφει πραγματικά το περιεχόμενό του και σημειώνει ότι το συγκεκριμένο δόλωμα αγγίχτηκε.

4. **Το εικονικό σύστημα αρχείων (VFS):** Παρουσιάζει στον επιτιθέμενο ένα φαινομενικά πλήρες σύστημα αρχείων, συνδυάζοντας δύο πηγές: τα decoy artefacts, τα οποία βρίσκονται σε πραγματικά αρχεία στον δίσκο (φάκελος decoys/fs/), και συνθετικά αρχεία συστήματος (όπως /etc/passwd, /etc/os-release, /proc/version) που παράγονται δυναμικά από το κοινό profile ώστε να είναι συνεπή. Το VFS περιλαμβάνει επίσης προστασία έναντι path traversal, εμποδίζοντας τον επιτιθέμενο να ξεφύγει εκτός του εικονικού περιβάλλοντος, και αντιμετωπίζει ειδικές περιπτώσεις, όπως το /etc/shadow που επιστρέφει permission denied χωρίς δικαιώματα root.

Αρχεία που υλοποιούν το Task 1: 02_ssh_honeypot.py (honeypot, shell emulator, VFS) και **00_config.py** (system profile). Το εικονικό σύστημα αρχείων βρίσκεται στο **decoys/fs/**.

Task 2: Παραγωγή decoy artefacts

Δημιουργήθηκαν δώδεκα (12) decoy artefacts σε επτά (7) κατηγορίες, ξεπερνώντας τα ελάχιστα όρια της εκφώνησης (10 artefacts, 3 κατηγορίες). Επτά (7) από αυτά παρήχθησαν με LLM, δηλαδή με το μοντέλο Claude, και πέντε (5) χειροκίνητα (template-based), καλύπτοντας την απαίτηση για τουλάχιστον πέντε LLM-generated artefacts.

Τα επτά δολώματα που παρήχθησαν με το μοντέλο Claude

Καθένα από τα επτά αυτά δολώματα σχεδιάστηκε ώστε να αποκαλύπτει σκόπιμα ένα κομμάτι της (πλασματικής) υποδομής, δίνοντας στον επιτιθέμενο πληροφορία που τον παρακινεί να εμβαθύνει:

- **Ιστορικό εντολών του λογαριασμού deployment (.bash_history του svc_deploy):** δείχνει τη ροή των αυτόματων εγκαταστάσεων (deployments) και αναφέρει τους εσωτερικούς κόμβους db01, cache01 και ci01. Λειτουργεί σαν χάρτης του πώς λειτουργεί το σύστημα.
- **nginx config (app.corp.conf):** αποκαλύπτει ότι η εφαρμογή τρέχει τοπικά στη θύρα 8000, τα μονοπάτια των πιστοποιητικών TLS και ένα εσωτερικό endpoint /metrics περιορισμένο στο εταιρικό δίκτυο.

- **Αρχείο ρυθμίσεων βάσης (database.yml):** αποκαλύπτει τους χρήστες της βάσης (app_rw με δικαιώματα εγγραφής, app_ro μόνο ανάγνωσης) και τον διακομιστή db01. Οι κωδικοί είναι εμφανώς placeholder, εγχεόμενοι κατά το deployment.
- **Log ελέγχου ταυτότητας (auth.log):** δείχνει ποιοι χρήστες συνδέθηκαν, από ποιον σταθμό διαχείρισης, καθώς και ενέργειες sudo και προγραμματισμένες εργασίες, ενισχύοντας την αίσθηση ενός ζωντανού συστήματος.
- **.ssh/config:** ίσως το πιο πολύτιμο δόλωμα, καθώς περιέχει έναν πλήρη χάρτη όλων των εσωτερικών μηχανημάτων (bastion, db01, cache01, backup01, ci01) και των κλειδιών πρόσβασης. Καθοδηγεί άμεσα προς οριζόντια μετακίνηση.
- **Runbook εγκατάστασης (deploy_runbook.md):** περιγράφει βήμα προς βήμα τη διαδικασία παραγωγής και επαναφοράς (rollback), αποκαλύπτοντας πλούσια πληροφορία για την αρχιτεκτονική και τις εξαρτήσεις της εφαρμογής.
- **Σημειώσεις υποδομής (NOTES_infra.md):** γραμμένες σαν βιαστικές ανθρώπινες σημειώσεις ενός διαχειριστή. Αποκαλύπτουν αδυναμίες, όπως έναν κωδικό που δεν έχει ακόμη αλλάξει, δημιουργώντας αίσθηση επείγοντος και ευκαιρίας.

Τα πέντε χειροκίνητα (template-based) δολώματα

Τα υπόλοιπα πέντε δημιουργήθηκαν χειροκίνητα και καλύπτουν πιο τυποποιημένα, αλλά εξίσου δελεαστικά στοιχεία:

- **Ιστορικό εντολών του διαχειριστή (.bash_history του mgeorgiou):** δείχνει τυπικές εργασίες διαχείρισης (έλεγχοι υπηρεσιών, σύνδεση στη βάση) και αναφορές σε άλλα εσωτερικά συστήματα.
- **Αρχείο ψεύτικων credentials (.env):** το πιο άμεσα δελεαστικό δόλωμα, καθώς μοιάζει να περιέχει μυστικά της εφαρμογής (διεύθυνση βάσης, κλειδιά, κωδικούς). Όλες όμως οι τιμές είναι εμφανώς placeholder (π.χ. CHANGE_ME, FAKE) και δείχνουν σε πλασματικούς κόμβους.
- **Log της PostgreSQL:** επιβεβαιώνει την έκδοση της βάσης (14) και το όνομα της βάσης δεδομένων, ενισχύοντας τη συνέπεια της ιστορίας.
- **Αρχείο γνωστών κόμβων SSH (.ssh/known_hosts):** αποκαλύπτει την εσωτερική τοπολογία, δηλαδή ποια μηχανήματα υπάρχουν και με ποιες διευθύνσεις, καθοδηγώντας κι αυτό προς οριζόντια μετακίνηση.

- **Προγραμματισμένες εργασίες (cron.d/deploy):** δείχνουν τους λογαριασμούς υπηρεσιών και τα scripts που εκτελούνται αυτόματα, στοιχεία που ένας επιτιθέμενος θα εξέταζε για σημεία μονιμότητας (persistence).

Artefact	Κατηγορία	Πηγή	Ρόλος στην ιστορία
.bash_history (mgeorgiou)	shell_history	manual	Ιστορικό εντολών sysadmin
.bash_history (svc_deploy)	shell_history	Claude	Ροή CI/CD deployment
app.corp.conf	config	Claude	nginx reverse proxy, /metrics
.env	credentials	manual	Ψεύτικα secrets (KEY=value)
database.yml	credentials	Claude	Χρήστες app_rw/app_ro, db01
postgresql-14-main.log	logs	manual	Δραστηριότητα PostgreSQL
auth.log	logs	Claude	Συνδέσεις SSH, sudo, cron
.ssh/known_hosts	ssh	manual	Εσωτερική τοπολογία δικτύου
.ssh/config	ssh	Claude	Χάρτης στόλου (db01, ci01 κ.α.)
cron.d/deploy	system	manual	Προγραμματισμένες εργασίες
deploy_runbook.md	docs	Claude	Runbook production deployment
NOTES_infra.md	docs	Claude	Σημειώσεις admin, αδυναμίες

Μέθοδος παραγωγής των LLM artefacts

Το περιεχόμενο των επτά LLM artefacts παρήχθη με το μοντέλο Claude, με ρητά prompts που περιέγραφαν το system profile (ονόματα, εκδόσεις, διευθύνσεις) και ζητούσαν ρεαλιστικό, εσωτερικά συνεπές περιεχόμενο χωρίς πραγματικά μυστικά. Κάθε prompt αποθηκεύεται αυτούσιο στον φάκελο llm_prompts/ για πλήρη τεκμηρίωση και αναπαραγωγιμότητα. Έτσι, το περιεχόμενο των LLM δολωμάτων είναι σταθερό και ενσωματωμένο, ώστε η εκτέλεση να είναι πάντα επαναλήψιμη.

Εσωτερική συνέπεια

Η εσωτερική συνέπεια ήταν συνειδητή και κεντρική σχεδιαστική επιλογή, καθώς οι αντιφάσεις είναι από τα πιο συχνά σημεία που προδίδουν ένα honeypot. Ένας προσεκτικός επιτιθέμενος συγκρίνει τις πληροφορίες που βρίσκει, και μια ασυμφωνία (για παράδειγμα, μια έκδοση λογισμικού που αναφέρεται διαφορετικά σε δύο αρχεία) αρκεί για να τον κάνει καχύποπτο.

Για την αποφυγή τέτοιων αντιφάσεων, όλα τα δολώματα αντλούν τα στοιχεία τους από μία ενιαία πηγή αλήθειας, το κοινό profile (00_config.py). Έτσι, ο ίδιος host (app-srv-01), οι ίδιοι χρήστες (mgeorgiou, svc_deploy), οι ίδιες εκδόσεις (OpenSSH 8.9p1, nginx 1.18.0, PostgreSQL 14) και το ίδιο εσωτερικό δίκτυο (10.20.0.0/16) εμφανίζονται με τον ίδιο ακριβώς τρόπο παντού. Επιπλέον, οι ημερομηνίες παράγονται από μια σταθερή ημερομηνία αναφοράς και όχι από την τρέχουσα ώρα, ώστε να μη μετακινούνται σε κάθε εκτέλεση και να μην αντιφάσκουν με τα κείμενα.

Πέρα από τη συνέπεια των επιμέρους στοιχείων, τα δολώματα συγκλίνουν σε μια ενιαία, εύλογη ιστορία που ο επιτιθέμενος μπορεί να ανασυνθέσει. Ενδεικτικά, η σημείωση στο NOTES_infra.md για την ανάγκη εναλλαγής του κωδικού του χρήστη app_rw αντιστοιχεί στον ίδιο χρήστη app_rw που εμφανίζεται στο αρχείο ρυθμίσεων βάσης database.yml. Το runbook εγκατάστασης (deploy_runbook.md) περιγράφει διαδικασία που ταιριάζει με τις εντολές που φαίνονται στο ιστορικό του λογαριασμού svc_deploy. Το αρχείο .ssh/config και το .ssh/known_hosts αναφέρουν τους ίδιους εσωτερικούς κόμβους (db01, cache01, backup01, ci01) με τις ίδιες διευθύνσεις. Αυτή η δικτυωμένη συνέπεια κάνει το περιβάλλον να μοιάζει με πραγματικό σύστημα με ιστορία, και όχι με μια συλλογή ασύνδετων ψεύτικων αρχείων.

Αρχεία που υλοποιούν το Task 2: 01_generate_decoys.py, ο φάκελος **decoys/** (τα artefacts και το artifact_inventory.json) και ο φάκελος **llm_prompts/** (τα prompts των LLM artefacts).

Task 3: Καταγραφή και ανακατασκευή συνεδριών

Η καταγραφή είναι το θεμέλιο κάθε honeypot: χωρίς ακριβή και δομημένα δεδομένα, καμία ανάλυση δεν είναι δυνατή. Στόχος του σταδίου αυτού είναι να καταγράφεται κάθε συνεδρία

με τέτοιο τρόπο ώστε να μπορεί αργότερα να ανακατασκευαστεί πλήρως, δηλαδή να μπορούμε να δούμε ακριβώς τι έκανε ο επιτιθέμενος, με ποια σειρά, και με ποιο αποτέλεσμα.

Δομή της καταγραφής

Η καταγραφή γίνεται ταυτόχρονα σε δύο μορφές, για δύο διαφορετικές χρήσεις. Πρώτον, κάθε συνεδρία αποθηκεύεται ως ένα αυτόνομο, ευανάγνωστο αρχείο JSON (logs/sessions/session_<id>.json), κατάλληλο για επιθεώρηση μιας μεμονωμένης συνεδρίας. Δεύτερον, όλες οι συνεδρίες προστίθενται σε ένα ενιαίο master αρχείο σε μορφή JSON Lines (logs/sessions.jsonl), όπου κάθε γραμμή είναι μια πλήρης συνεδρία. Η μορφή αυτή είναι ιδανική για μαζική ανάλυση, καθώς το πρόγραμμα ανάλυσης διαβάζει το αρχείο γραμμή προς γραμμή.

Για κάθε συνεδρία καταγράφονται η διεύθυνση και η θύρα του πελάτη, οι χρονικές σφραγίδες έναρξης και λήξης (και άρα η διάρκεια), το banner του SSH client, όλες οι προσπάθειες authentication, η πλήρης ακολουθία εντολών, και η λίστα με τα decoy artefacts που αγγίχτηκαν. Επιπλέον, κάθε εντολή καταγράφεται ως ξεχωριστό συμβάν (event) με τον αύξοντα αριθμό της, τη χρονική σφραγίδα, τον τρέχοντα κατάλογο τη στιγμή της εκτέλεσης, το μέγεθος της απόκρισης, και τη λίστα των artefacts που άγγιξε η συγκεκριμένη εντολή. Το πλήρες σχήμα τεκμηριώνεται αναλυτικά στο αρχείο SCHEMA.md.

Παρακολούθηση των αγγιγμάτων (artefact tracking)

Το σημαντικότερο στοιχείο της καταγραφής είναι η παρακολούθηση των αγγιγμάτων. Κάθε φορά που μια εντολή διαβάζει ένα δολώμα, το honeypot το σημειώνει. Έτσι, μια εντολή όπως `grep -r password /opt/app` μπορεί να αγγίξει ταυτόχρονα δύο δολώματα (το `.env` και το `database.yml`), και αυτό καταγράφεται με ακρίβεια. Χάρη σε αυτό, η μετρική *artefacts touched* γίνεται πραγματικό, μετρήσιμο μέγεθος, και όχι εικασία. Αυτό είναι που επιτρέπει, στο επόμενο στάδιο, να μετρήσουμε αντικειμενικά πόσο βαθιά ενεπλάκη ο κάθε επιτιθέμενος με το περιβάλλον.

Στην πραγματική εκτέλεση καταγράφηκαν 55 συνεδρίες συνολικά: 35 συνεδρίες με εντολές (τα playbooks) και 20 συνεδρίες αποτυχημένης authentication. Οι τελευταίες αντιστοιχούν στις προσωπικότητες (personas) που δοκιμάζουν πρώτα έναν λανθασμένο κωδικό πριν συνδεθούν. Η καταγραφή και αυτών των αποτυχημένων προσπαθειών είναι σημαντική, καθώς αποτελεί ρεαλιστικό σήμα credential probing, δηλαδή προσπάθειας εύρεσης κωδικού.

Αρχεία που υλοποιούν το Task 3: 02_ssh_honeyrot.py (η καταγραφή), ο φάκελος logs/ (sessions/ και sessions.jsonl) και το SCHEMA.md (η πλήρης τεκμηρίωση του σχήματος).

Task 4: Ελεγχόμενα playbooks επιτιθεμένου

Ένα playbook είναι ένα προκαθορισμένο σενάριο επίθεσης, δηλαδή μια συγκεκριμένη ακολουθία εντολών που εκτελεί ένας επιτιθέμενος με έναν σαφή στόχο. Για παράδειγμα, ένα playbook αναζήτησης credentials θα εκτελέσει εντολές που ψάχνουν για κωδικούς και κλειδιά. Επειδή δεν διαθέταμε πραγματικούς επιτιθεμένους, τα playbooks μας παίζουν ελεγχόμενα τον ρόλο τους, ώστε να παραχθεί ρεαλιστική και ποικίλη κίνηση προς το honeyrot. Ορίστηκαν επτά (7) playbooks, καθένα με διακριτό στόχο, τα οποία μαζί καλύπτουν το φάσμα των τακτικών ενός επιτιθεμένου μετά την αρχική πρόσβαση.

Playbook	Στόχος (intent)	Τι κάνει
recon	reconnaissance	Βασική αναγνώριση συστήματος και χρήστη
credential_search	credential_harvesting	Αναζήτηση κωδικών και κλειδιών
config_discovery	environment_mapping	Χαρτογράφηση υπηρεσιών μέσω configs
privilege_escalation	privilege_escalation	Προσπάθεια ανόδου προνομίων (sudo, shadow)
data_staging	data_exfiltration	Εντοπισμός δεδομένων προς εξαγωγή
lateral_movement	lateral_movement	Προετοιμασία μετάβασης σε άλλα μηχανήματα
honeyrot_fingerprinting	honeyrot_detection	Έλεγχος για sandbox/honeyrot

Παραλλαγές βάθους: ρηχές και βαθιές συνεδρίες

Κάθε playbook εκτελείται σε τρεις παραλλαγές βάθους, ώστε να προσομοιωθεί το γεγονός ότι στην πραγματικότητα άλλοι επιτιθέμενοι είναι βιαστικοί και επιφανειακοί, ενώ άλλοι είναι μεθοδικοί και εξονυχιστικοί. Η **ρηχή (shallow)** παραλλαγή εκτελεί μόνο τις πρώτες, βασικές εντολές, σαν έναν επιτιθέμενο που ρίχνει μια γρήγορη ματιά. Η **βαθιά (deep)** παραλλαγή εκτελεί όλη την ακολουθία, διαβάζοντας και τα δολώματα. Η **βαθύτερη (deep_plus)** παραλλαγή προσθέτει και επιπλέον εντολές follow-up, σαν έναν επιτιθέμενο που ακολουθεί τα στοιχεία που βρίσκει. Για παράδειγμα, στο playbook αναζήτησης credentials, η ρηχή εκδοχή απλώς κοιτάζει τον προσωπικό φάκελο, ενώ η βαθιά διαβάζει το .env, το database.yml και το .ssh/config.

Σε κάθε εκτέλεση προστίθεται και ελαφριά τυχαιότητα (μικρές διαφορές στη σειρά, στον αριθμό των εντολών και στις παύσεις), ώστε επαναλήψεις του ίδιου playbook να μη βγαίνουν πανομοιότυπες. Έτσι παρήχθη ένα σύνολο 35 συνεδριών με πραγματική ποικιλία βάθους. Η εκφώνηση απαιτεί τουλάχιστον 15 συνεδρίες, οπότε το όριο ξεπερνιέται σκόπιμα, για ουσιαστικότερη στατιστική ανάλυση και clustering.

Γιατί κρατάμε το ground truth ξεχωριστά

Ο όρος ground truth σημαίνει την επιβεβαιωμένη, γνωστή αλήθεια. Επειδή εμείς οι ίδιοι ορίζουμε ποιο playbook τρέχει κάθε φορά, γνωρίζουμε με βεβαιότητα ποιος ήταν ο πραγματικός στόχος (intent) κάθε συνεδρίας. Αυτή η γνωστή αλήθεια αποθηκεύεται σε ξεχωριστό αρχείο (playbook_results/ground_truth.json).

Η σημαντική μεθοδολογική λεπτομέρεια είναι ότι το honeypot δεν γνωρίζει αυτόν τον στόχο. Το honeypot καταγράφει μόνο τι συνέβη, δηλαδή ποιες εντολές δόθηκαν, χωρίς καμία ετικέτα intent. Έτσι, στο Task 5 η ομαδοποίηση των συνεδριών γίνεται στα τυφλά, μόνο πάνω στα δεδομένα συμπεριφοράς, και μόνο στο τέλος συγκρίνεται με το ground truth για να δούμε πόσο καλά η ανάλυση ανακάλυψε μόνη της τις πραγματικές τακτικές. Αν αντίθετα το intent προερχόταν από το ίδιο το όνομα του playbook, η ανάλυση θα ήταν ταυτολογική, δηλαδή θα επιβεβαίωνε κάτι που ήδη γνωρίζαμε.

Αρχεία που υλοποιούν το Task 4: 03_run_playbooks.py και ο φάκελος playbook_results/ (ground_truth.json, playbook_catalogue.json).

Task 5: Ανάλυση συμπεριφοράς και πρόθεσης

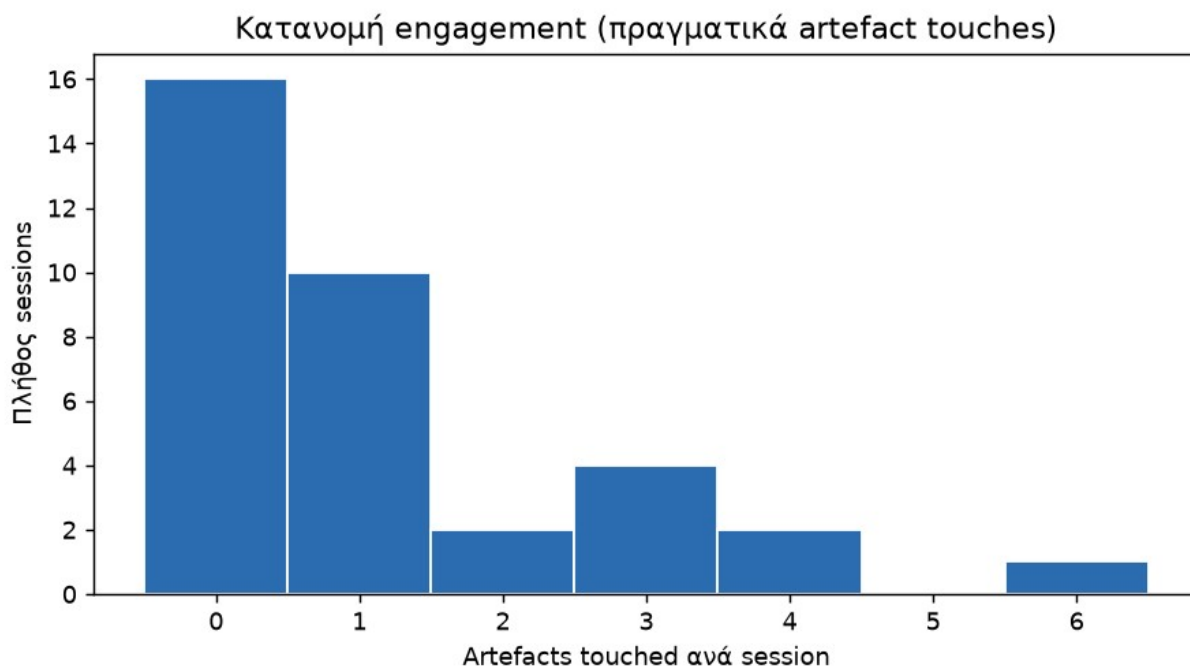
Στόχος του σταδίου αυτού είναι να μετατρέψουμε τα ακατέργαστα logs σε ουσιαστική γνώση για τη συμπεριφορά των επιτιθεμένων. Η ανάλυση γίνεται σε τρία βήματα: πρώτα εξάγουμε αριθμητικά χαρακτηριστικά (features) από κάθε συνεδρία, μετά ομαδοποιούμε τις συνεδρίες με βάση αυτά, και τέλος ξεχωρίζουμε τις πιο βαθιές, στοχευμένες συνεδρίες.

Χαρακτηριστικά συμπεριφοράς (features)

Για κάθε συνεδρία με εντολές εξήχθησαν οκτώ χαρακτηριστικά. Πέντε από αυτά μετρούν την ένταση της εμπλοκής: τη διάρκεια (dwell time), το πλήθος εντολών, την ποικιλία εντολών (μοναδικές προς συνολικές), το πλήθος των artefacts που αγγίχτηκαν, και το μέγιστο βάθος διαδρομής (path depth). Τα άλλα τρία μετρούν το είδος της δραστηριότητας, ως αναλογίες: το ποσοστό εντολών ανάγνωσης αρχείων, το ποσοστό εντολών αναγνώρισης συστήματος, και το ποσοστό εντολών αναβάθμισης προνομίων. Ο συνδυασμός έντασης και είδους επιτρέπει τον διαχωρισμό όχι μόνο του πόσο βαθιά, αλλά και του τι είδους ενέργειες έκανε κάθε επιτιθέμενος.

Μετρική	Μέσος όρος	Περιγραφή
dwell_time	5.43 s	Διάρκεια συνεδρίας
command_count	5.20	Πλήθος εντολών
cmd_diversity	0.83	Μοναδικές/συνολικές εντολές
artefacts_touched	1.14	Decoys που διαβάστηκαν
path_depth	2.91	Μέγιστο βάθος διαδρομής

Το πρώτο και σημαντικότερο εύρημα είναι ότι η μετρική artefacts_touched είναι πραγματική. Σε ένα ελάχιστο στατικό περιβάλλον χωρίς δολώματα, καμία εντολή ανάγνωσης δεν θα επέστρεφε περιεχόμενο, οπότε η τιμή θα ήταν 0 σε όλες τις συνεδρίες. Η παρουσία των δολωμάτων προκάλεσε πραγματική εμπλοκή, με μέσο όρο 1.14 και μέγιστο 6 artefacts σε μία συνεδρία. Το παρακάτω σχήμα δείχνει την κατανομή: αρκετές συνεδρίες (κυρίως οι ρηχές) δεν άγγιξαν κανένα δόλωμα, ενώ ένα σαφές υποσύνολο βαθιών συνεδριών άγγιξε τρία ή περισσότερα. Αυτή η διασπορά είναι ακριβώς αυτό που κάνει την ανάλυση ενδιαφέρουσα.



Ομαδοποίηση των συνεδριών (clustering)

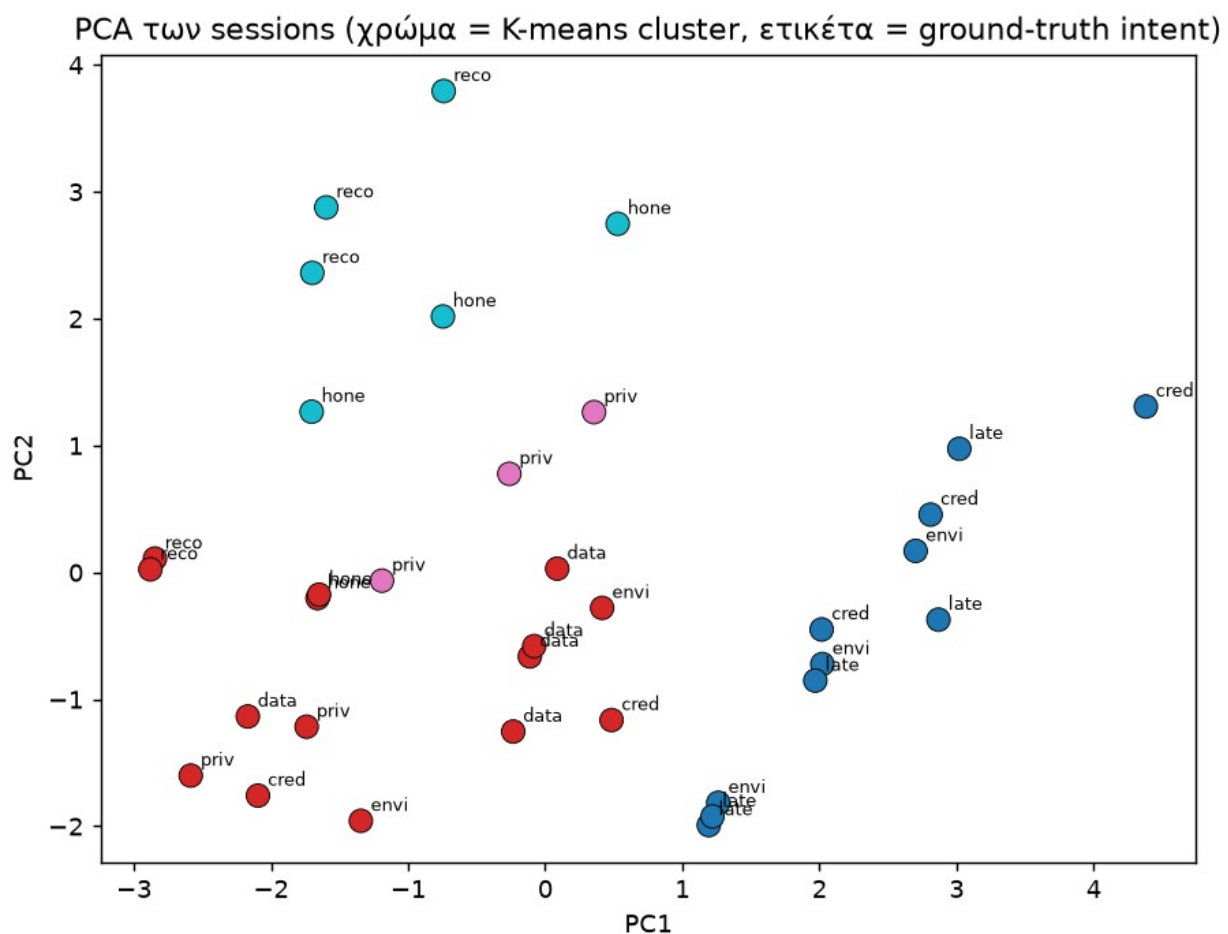
Για να ανακαλύψουμε τα μοτίβα συμπεριφοράς χωρίς προκαταλήψεις, εφαρμόσαμε δύο μη επιβλεπόμενες μεθόδους πάνω στα κανονικοποιημένα χαρακτηριστικά: K-means και DBSCAN (διάλεξη 3). Το K-means ζητά εκ των προτέρων τον αριθμό των ομάδων. Επιλέξαμε τέσσερις ομάδες ($k=4$), τιμή που δοκιμάστηκε με τον δείκτη silhouette και δίνει την πιο καθαρή και ερμηνεύσιμη διάκριση προφίλ. Η ομαδοποίηση ανέδειξε τέσσερα σαφή προφίλ συμπεριφοράς.

Προφίλ ομάδας	Μέγεθος	Μ.ο. touches	Μ.ο. εντολών
Βαθιοί αναγνώστες (deep readers)	11	2.73	5.64
Ρηχά probes (shallow)	15	0.60	3.67
Βαθιά αναγνώριση χωρίς ανάγνωση	6	0.00	7.83
Ενδιάμεσο probing	3	0.33	6.00

Η ερμηνεία είναι άμεσα αξιοποιήσιμη για ένα κέντρο επιχειρήσεων ασφάλειας (SOC). Οι βαθιοί αναγνώστες είναι η πιο επικίνδυνη ομάδα, καθώς διαβάζουν πολλά δολώματα (κατά μέσο όρο 2.73), δηλαδή αναζητούν ενεργά ευαίσθητη πληροφορία. Η βαθιά αναγνώριση εκτελεί τις

περισσότερες εντολές (7.83) αλλά δεν διαβάζει δολώματα, μοτίβο τυπικό του recon και του fingerprinting, όπου ο επιτιθέμενος εξετάζει το σύστημα (ps, uname, /proc) χωρίς να ανοίγει αρχεία. Τα ρηχά probes είναι η μεγαλύτερη αλλά λιγότερο επικίνδυνη ομάδα, με επιφανειακή εξερεύνηση.

Το παρακάτω σχήμα προβάλλει τις συνεδρίες σε δύο διαστάσεις με τη μέθοδο PCA (διάλεξη 3), η οποία συμπυκνώνει τα οκτώ χαρακτηριστικά σε δύο άξονες για οπτικοποίηση. Το χρώμα δείχνει την ομάδα K-means και η ετικέτα το πραγματικό intent. Παρατηρείται ότι οι συνεδρίες απλώνονται από αριστερά (ρηχές) προς δεξιά (βαθείς), και ότι οι ομάδες σχηματίζουν διακριτές περιοχές.

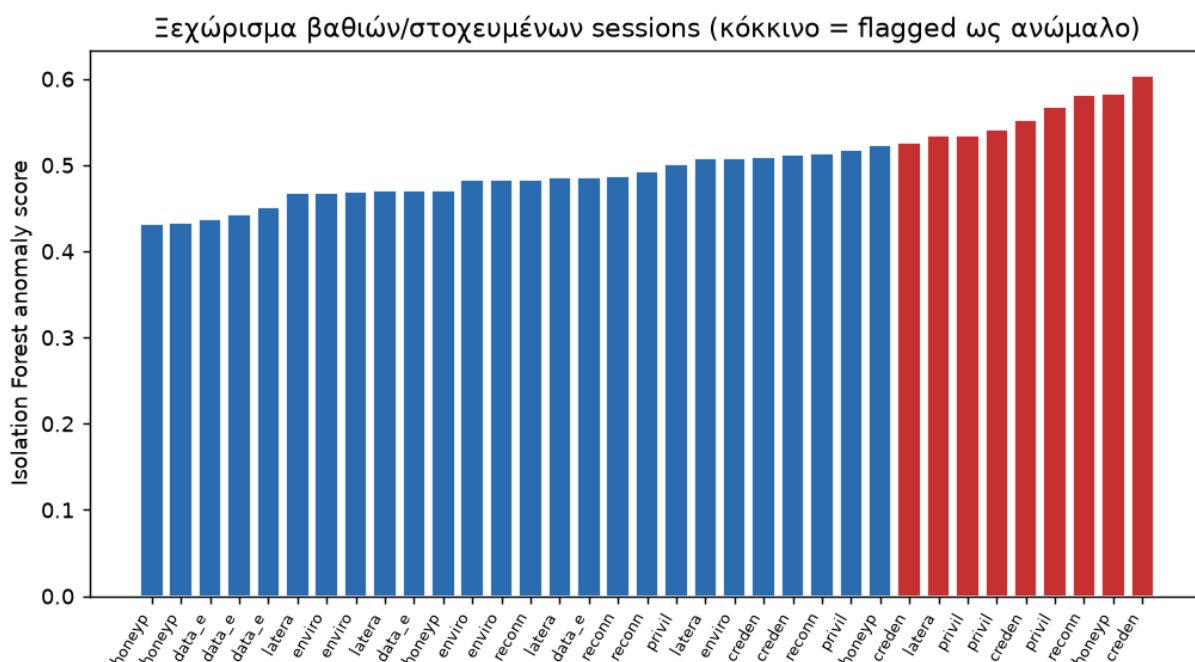


Συγκρίνοντας τις ομάδες με το ground truth, ο δείκτης Adjusted Rand Index (ARI) είναι χαμηλός (0.185). Ο δείκτης αυτός μετρά πόσο οι ομάδες συμπίπτουν με τα πραγματικά intents, με το 1.0 να σημαίνει τέλεια συμφωνία. Η χαμηλή τιμή **δεν είναι αποτυχία αλλά εύρημα**: τα χαρακτηριστικά συμπεριφοράς ομαδοποιούν τις συνεδρίες κατά βάθος και προφίλ εμπλοκής, όχι κατά το ακριβές σημασιολογικό intent. Τακτικές με παρόμοιο προφίλ (για παράδειγμα το recon και το fingerprinting, που και τα δύο εκτελούν πολλές εντολές αναγνώρισης χωρίς να διαβάζουν δολώματα) πέφτουν λογικά στην ίδια ομάδα. Το ερώτημα αν μπορούμε να προβλέψουμε το

ακριβές intent αντιμετωπίζεται με επιβλεπόμενο μοντέλο στο προαιρετικό task, με σαφώς καλύτερα αποτελέσματα.

Ανίχνευση βαθιών συνεδριών (Isolation Forest)

Τέλος, χρησιμοποιήσαμε Isolation Forest (διάλεξη 3), μια μέθοδο ανίχνευσης ανωμαλιών, για να ξεχωρίσουμε αυτόματα τις βαθιές, στοχευμένες συνεδρίες από τις ρηχές, καθημερινές. Το μοντέλο σημείωσε 9 από τις 35 συνεδρίες ως ανώμαλες. Όπως δείχνει το **Σχήμα 3**, αυτές αντιστοιχούν στις πιο βαθιές αλληλεπιδράσεις (credential search, privilege escalation, deep recon), δηλαδή ακριβώς εκείνες που θα ήθελε να δει πρώτος ένας αναλυτής ασφάλειας. Πρακτικά, το Isolation Forest λειτουργεί ως ένα αυτόματο φίλτρο προτεραιότητας.



Ενδεικτικές συνόψεις συνεδριών

Ακολουθούν τρεις αντιπροσωπευτικές συνόψεις, βασισμένες σε πραγματικά δεδομένα των logs, μία από κάθε βασικό προφίλ:

- **Βαθύς αναγνώστης (credential_harvesting, deep_plus):** 9 εντολές σε 11.9s, βάθος διαδρομής 3, άγγιξε 6 δολώματα (.env, database.yml, .ssh/config, known_hosts, bash_history, auth.log). Σαφής πρόθεση συλλογής credentials και χαρτογράφησης του δικτύου.

- **Βαθιά αναγνώριση (honeypot_detection, deep):** 8 εντολές, 0 δολώματα. Εκτέλεσε `cat /proc/version, ls /.dockerenv, ps aux, ip addr`, δηλαδή ελέγχους περιβάλλοντος για να διαπιστώσει αν βρίσκεται σε sandbox, χωρίς να ανοίξει αρχεία.
- **Πηχό probe (reconnaissance, shallow):** 3 εντολές σε 3s (`whoami, id, uname`). Επιφανειακή αναγνώριση, καμία εμπλοκή με δολώματα.

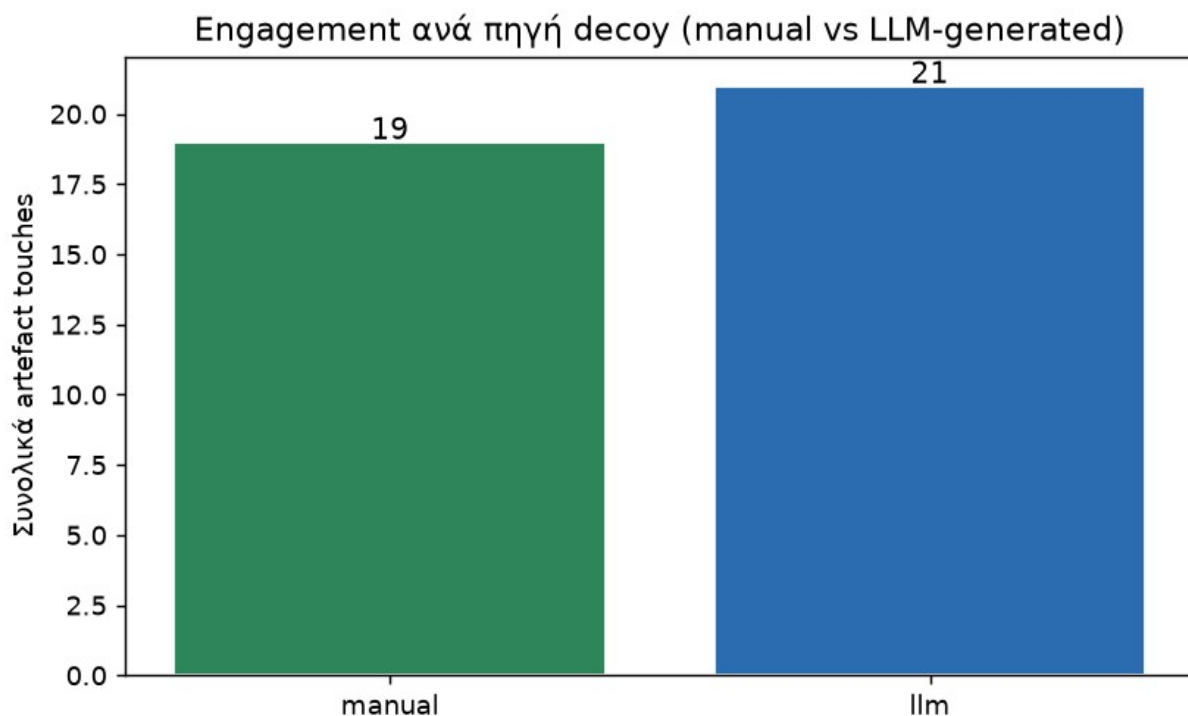
Αρχεία που υλοποιούν το Task 5: `04_analyze_sessions.py`, ο φάκελος `analysis/` (`session_features.csv`, `analysis_report.json`, `cluster_composition.json`) και ο φάκελος `figures/`.

Task 6: Ποιότητα παραπλάνησης και κίνδυνος fingerprinting

Το στάδιο αυτό αξιολογεί δύο πράγματα: πόσο πειστικά είναι τα δολώματα (και ειδικά αν τα LLM-generated αποδίδουν καλύτερα από τα χειροκίνητα), και πόσο εύκολα μπορεί ένας επιτιθέμενος να καταλάβει ότι βρίσκεται σε honeypot.

Χειροκίνητα έναντι LLM-generated δολωμάτων

Για να συγκρίνουμε τις δύο πηγές, μετρήσαμε πόσο engagement τράβηξε η καθεμία, χρησιμοποιώντας τα πραγματικά artefact touches. Τα χειροκίνητα δολώματα δέχτηκαν 19 αγγίγματα και τα LLM-generated 21. Το εύρημα είναι ότι τα LLM-generated δολώματα δεν υστερούν σε engagement έναντι των χειροκίνητων, όπως δείχνει και το Σχήμα 4. Χρειάζεται ωστόσο να σημειωθεί ότι τα αγγίγματα εξαρτώνται και από το ποια μονοπάτια στοχεύουν τα playbooks, όχι μόνο από την ελκυστικότητα του περιεχομένου, οπότε η σύγκριση δείχνει ισοδυναμία μάλλον παρά υπεροχή.



Ποιοτικά, τα LLM-generated δολώματα (το runbook, οι σημειώσεις υποδομής, το ssh config) είναι πλουσιότερα σε αφήγηση και εσωτερική συνέπεια, ενώ τα χειροκίνητα είναι συντομότερα και πιο τυποποιημένα. Κατά κατηγορία, τα δολώματα ssh (known_hosts, config) δέχτηκαν το περισσότερο engagement (11 αγγίγματα), ακολουθούμενα από τα credentials (8) και το ιστορικό εντολών (6), γεγονός αναμενόμενο, δεδομένου ότι αυτά αποκαλύπτουν τοπολογία δικτύου και προσβάσεις, δηλαδή ό,τι ψάχνει πρώτα ένας επιτιθέμενος.

Πώς μπορεί να προδοθεί το honeypot (fingerprinting)

Για λόγους πληρότητας, καταγράφονται τα σημεία στα οποία ένας προσεκτικός επιτιθέμενος θα μπορούσε να αντιληφθεί ότι βρίσκεται σε honeypot:

- Ρηχό σύστημα αρχείων: το εικονικό filesystem περιέχει μόνο τα δολώματα και λίγα συνθετικά αρχεία. Ένα εκτεταμένο find / ή ls σε μη προβλεπόμενους καταλόγους αποκαλύπτει την έλλειψη βάθους.
- Περιορισμένο σύνολο εντολών: εντολές εκτός του υποστηριζόμενου υποσυνόλου επιστρέφουν command not found. Ένας επιτιθέμενος που δοκιμάζει σπάνιες εντολές θα εντοπίσει ασυνέπειες.
- Ομοιόμορφος χρόνος απόκρισης: οι αποκρίσεις είναι σχεδόν ακαριαίες και χωρίς ρεαλιστικό latency, κάτι που διαφέρει από ένα πραγματικό σύστημα υπό φορτίο.

- Απουσία πλήρους τερματικού (PTY): δεν υποστηρίζονται tab completion, job control και πλήρης χειρισμός χρωμάτων/ANSI, στοιχεία που ένας έμπειρος χρήστης αναμένει.
- Στατικά δολώματα και σταθερό κλειδί: το περιεχόμενο δεν αλλάζει μεταξύ συνδέσεων. Περιοδική αναγέννηση (dynamic refresh) θα μείωνε αυτό το σημάδι.

Ένα επιπλέον, γενικότερο ρίσκο των LLM-generated δολωμάτων είναι οι λεγόμενες hallucinations, δηλαδή η πιθανότητα το μοντέλο να εισαγάγει μια λεπτομέρεια που αντιφάσκει με το υπόλοιπο περιβάλλον. Στην παρούσα εργασία το ρίσκο αυτό μετριάστηκε τροφοδοτώντας όλα τα prompts με το ενιαίο system profile και ελέγχοντας τη συνέπεια του παραγόμενου περιεχομένου.

Αρχεία που υλοποιούν το Task 6: 04_analyze_sessions.py (ανάλυση manual vs LLM, analysis_report.json).

Προαιρετικό Task: Supervised ταξινομητής πρόθεσης

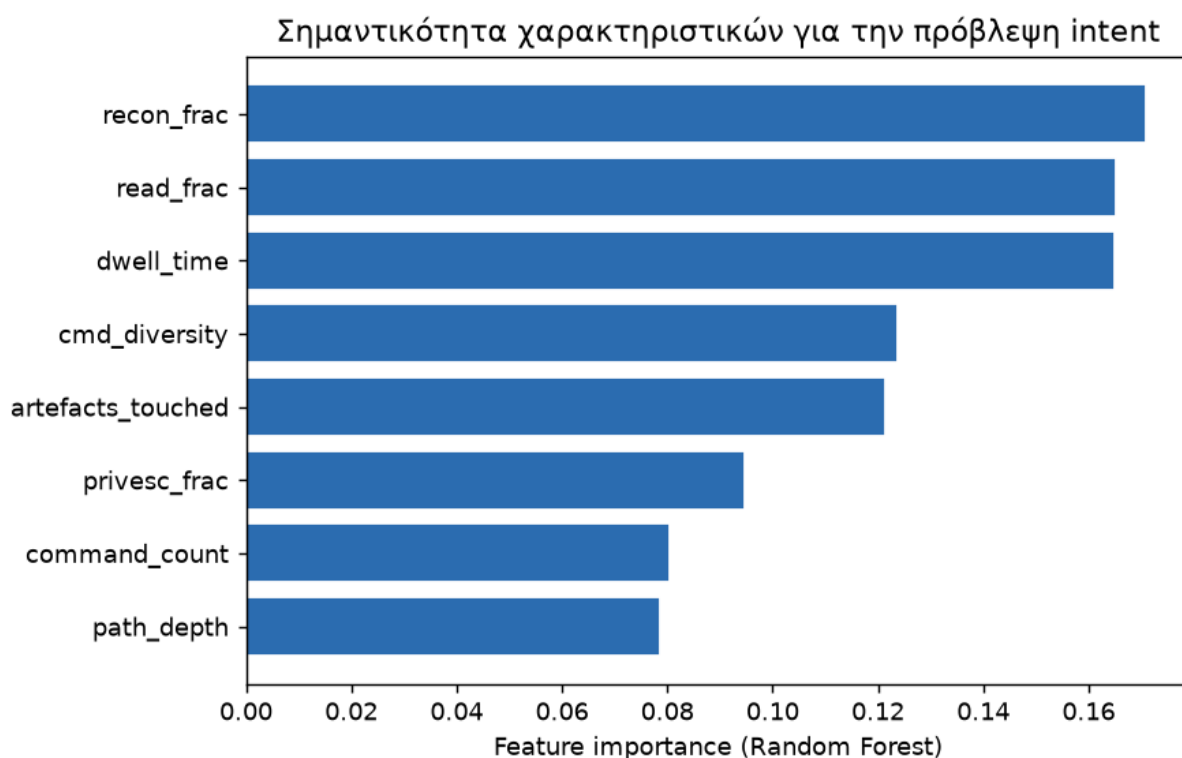
Από τα προαιρετικά προχωρημένα tasks της εκφώνησης υλοποιήθηκε ένα πλήρες και αυτοτελές: ένας supervised ταξινομητής πρόθεσης. Επιπλέον, η σύγκριση χειροκίνητων έναντι LLM-generated δολωμάτων (Task 6) αποτελεί και αυτή ένα από τα προτεινόμενα προχωρημένα θέματα, το οποίο καλύφθηκε με πραγματικά δεδομένα.

Το κίνητρο του ταξινομητή προκύπτει άμεσα από το Task 5: εκεί, το unsupervised clustering ομαδοποίησε τις συνεδρίες κατά προφίλ εμπλοκής και όχι κατά ακριβές intent (χαμηλό ARI). Το ερώτημα που μένει είναι αν το intent είναι καθόλου προβλέψιμο από τη συμπεριφορά. Η απάντηση, με επιβλεπόμενη μάθηση, είναι θετική.

Χρησιμοποιήθηκε ένα μοντέλο Random Forest (διάλεξη 3), το οποίο μαθαίνει από παραδείγματα να αντιστοιχίζει τα χαρακτηριστικά μιας συνεδρίας στο intent της. Επειδή το σύνολο δεδομένων είναι μικρό (35 συνεδρίες, 7 κλάσεις), η αξιολόγηση έγινε με τη μέθοδο Leave-One-Out Cross-Validation: κάθε συνεδρία προβλέπεται από ένα μοντέλο εκπαιδευμένο στις υπόλοιπες 34, ώστε καμία πρόβλεψη να μη γίνεται πάνω σε δεδομένα που το μοντέλο έχει ήδη δει. Το μοντέλο πέτυχε ακρίβεια 0.571 και macro-F1 0.559, σαφώς πάνω από την τυχαία πρόβλεψη, που για 7 κλάσεις θα ήταν περίπου 0.14. Δηλαδή, το μοντέλο προβλέπει σωστά το intent στις έξι από τις δέκα περιπτώσεις.

Το παρακάτω σχήμα δείχνει τη σημαντικότητα των χαρακτηριστικών. Τα πιο καθοριστικά είναι το ποσοστό εντολών αναγνώρισης (recon_frac), το ποσοστό εντολών ανάγνωσης (read_frac) και

η διάρκεια (dwell_time). Αυτό είναι διαισθητικά λογικό: το είδος των εντολών που δίνει ένας επιτιθέμενος (αν ψάχνει σύστημα ή διαβάζει αρχεία) φανερώνει τον στόχο του. Το συμπέρασμα είναι ότι τα χαρακτηριστικά συμπεριφοράς φέρουν πραγματική πληροφορία intent, την οποία το unsupervised clustering δεν μπορούσε να αξιοποιήσει πλήρως χωρίς ετικέτες, αλλά ο supervised ταξινομητής την εκμεταλλεύεται.



Αρχεία που υλοποιούν το προαιρετικό task: 05_intent_classifier.py και το analysis/intent_classifier.json.

Task 7: Ηθική, ασφάλεια και επιχειρησιακή χρήση

Η κατασκευή ενός honeypot, και ιδιαίτερα η χρήση LLM για την παραγωγή πειστικού παραπλανητικού περιεχομένου, εγείρει ζητήματα ηθικής που αντιμετωπίζονται με βάση τους πυλώνες της υπεύθυνης τεχνητής νοημοσύνης (διάλεξη 7).

Απομόνωση και ασφάλεια

Όλο το σύστημα λειτουργεί αποκλειστικά σε απομονωμένο εργαστήριο, με το honeypot να ακούει μόνο στο loopback (127.0.0.1). Δεν εκτίθεται τίποτα στο διαδίκτυο, ώστε το honeypot

να μη μπορεί να χρησιμοποιηθεί ως σημείο αναπήδησης (pivot) για επιθέσεις σε τρίτους, κάτι που θα το μετέτρεπε από εργαλείο άμυνας σε κίνδυνο.

Ελαχιστοποίηση δεδομένων

Δεν χρησιμοποιήθηκαν πραγματικά credentials, προσωπικά δεδομένα ή ζωντανά μυστικά. Όλες οι διευθύνσεις ανήκουν σε ιδιωτικές ζώνες ή ζώνες τεκμηρίωσης (RFC 1918 και RFC 5737) και όλα τα hosts είναι πλασματικά. Στην καταγραφή, δεν αποθηκεύονται ποτέ ολόκληρα credentials, παρά μόνο ένα δείγμα, ώστε ακόμη και τα ίδια τα logs να μην αποτελούν κίνδυνο διαρροής.

Η ηθική διάσταση της χρήσης LLM για παραπλάνηση

Το ερώτημα αν η χρήση LLM για παραγωγή παραπλανητικού υλικού αλλάζει το ηθικό προφίλ είναι ουσιώδες, και η απάντηση είναι θετική σε τρία σημεία. Πρώτον, η κλίμακα: ένα LLM μπορεί να παράγει απεριόριστο, ποικίλο και πειστικό περιεχόμενο με ελάχιστο κόστος, κάτι αδύνατο χειροκίνητα. Δεύτερον, η πειστικότητα: το υλικό είναι δυσκολότερο να ξεχωριστεί από το πραγματικό, άρα πιο αποτελεσματικό αλλά και πιο επικίνδυνο αν στραφεί προς κακή χρήση. Τρίτον, η διπλή χρήση: η ίδια ακριβώς τεχνική που παράγει δολώματα για άμυνα μπορεί να παράγει υλικό για phishing ή social engineering. Για τους λόγους αυτούς, η χρήση περιορίστηκε αυστηρά σε αμυντικό, απομονωμένο πλαίσιο, με πλήρως τεκμηριωμένα prompts και χωρίς παραγωγή υλικού που θα μπορούσε να στοχοποιήσει πραγματικά πρόσωπα ή συστήματα.

Επιχειρησιακή χρήση και μετάβαση σε παραγωγή

Για μια πραγματική ανάπτυξη, το σύστημα θα απαιτούσε τέσσερα πράγματα: ενισχυμένη απομόνωση (containerization και αυστηρούς περιορισμούς εξόδου στο δίκτυο), ανθρώπινη εποπτεία (human-in-the-loop) στην ανάλυση των ευρημάτων, σαφή νομική και οργανωτική έγκριση, και περιοδική αναγέννηση των δολωμάτων ώστε να μη γίνονται προβλέψιμα. Ο κύκλος διαχείρισης κινδύνου (identify, assess, mitigate, monitor) πρέπει να είναι συνεχής, όχι μια εφάπαξ ενέργεια.

Αρχεία που τεκμηριώνουν το Task 7: η παρούσα αναφορά, το **README.txt** (σημειώσεις ασφαλείας) και τα σχόλια εντός των **00_config.py** και **02_ssh_honeypot.py**.

Συνθήκες και παράμετροι εκτέλεσης

- Επαναληψιμότητα: σταθερός σπόρος τυχειότητας (25012) στην παραγωγή δολωμάτων και στα playbooks.
- Περιβάλλον: Python 3.10 και νεότερο (δοκιμασμένο σε 3.10 και 3.12), με τις βιβλιοθήκες paramiko, scikit-learn, numpy, pandas και matplotlib. Λειτουργεί σε Linux (Kali) και σε Windows.
- Δίκτυο: το honeypot ακούει στο 127.0.0.1:2222. Όλη η κίνηση είναι τοπική.
- Εκτέλεση: όλη η αλυσίδα τρέχει με μία εντολή (python3 run_all.py), όπως τεκμηριώνεται στο README.txt.

Περιορισμοί και απειλές εγκυρότητας

Για την ορθή ερμηνεία των αποτελεσμάτων, δηλώνονται τα όρια εντός των οποίων ισχύουν τα συμπεράσματα:

- Συνθετικοί επιτιθέμενοι: η κίνηση παράγεται από προκαθορισμένα playbooks, όχι από πραγματικούς αντιπάλους. Οι πραγματικές συμπεριφορές θα ήταν πιο ποικίλες και απρόβλεπτες.
- Μικρό δείγμα: 35 συνεδρίες είναι αρκετές για επίδειξη, αλλά οι μετρικές του ταξινομητή είναι ενδεικτικές και θα σταθεροποιούνταν με περισσότερα δεδομένα.
- Περιορισμένο κέλυφος: ο προσομοιωτής καλύπτει ένα ρεαλιστικό αλλά όχι πλήρες υποσύνολο εντολών, γεγονός που δημιουργεί τα σημεία fingerprinting του Task 6.
- Στατικά δολώματα: η έλλειψη δυναμικής αναγέννησης καθιστά το περιβάλλον προβλέψιμο σε επαναλαμβανόμενες συνδέσεις.

Συμπέρασμα

Κατασκευάστηκε ένα ολοκληρωμένο και επαναλήψιμο πλαίσιο honeypot analytics, το οποίο συνδυάζει ένα πραγματικό SSH honeypot με προσομοίωση κελύφους, δολώματα εμπλουτισμένα με LLM, δομημένη καταγραφή και ανάλυση συμπεριφοράς βασισμένη σε μη επιβλεπόμενη και επιβλεπόμενη μάθηση. Η κεντρική σχεδιαστική αρχή ήταν ότι η ανάλυση πρέπει να βασίζεται σε πραγματικά γεγονότα: επειδή το honeypot σερβίρει πραγματικά τα δολώματα, η μετρική `artefacts_touched` είναι πραγματική και όχι εικαζόμενη.

Τρία ευρήματα συνοψίζουν την εργασία. Πρώτον, τα LLM-generated δολώματα προκάλεσαν engagement συγκρίσιμο με τα χειροκίνητα (21 έναντι 19 αγγίγματα), οπότε αποτελούν βιώσιμη επιλογή για κλιμάκωση της παραγωγής δολωμάτων. Δεύτερον, το unsupervised clustering ομαδοποιεί αποτελεσματικά τις συνεδρίες κατά προφίλ εμπλοκής (βαθιοί αναγνώστες, ρηχά probes, βαθιά αναγνώριση), κάτι άμεσα αξιοποιήσιμο για την προτεραιοποίηση σε ένα SOC, ενώ η πρόβλεψη του ακριβούς intent απαιτεί επιβλεπόμενο μοντέλο, το οποίο πέτυχε ακρίβεια 0.571. Τρίτον, η καταγραφή των σημείων fingerprinting και των ηθικών ζητημάτων είναι αναπόσπαστο μέρος μιας υπεύθυνης προσέγγισης. Συνολικά, η εργασία δείχνει ότι η ενίσχυση με LLM αυξάνει την πειστικότητα ενός honeypot, αλλά η αξία του τελικά εξαρτάται από την αυστηρότητα της καταγραφής και την ακρίβεια της ανάλυσης.

Παραδοτέα και αντιστοίχιση Task με αρχεία

Task	Υλοποιείται από
1. Σχεδίαση honeypot	02_ssh_honeypot.py, 00_config.py
2. Decoy artefacts	01_generate_decoys.py, decoys/, llm_prompts/
3. Καταγραφή συνεδριών	02_ssh_honeypot.py, logs/, SCHEMA.md
4. Attacker playbooks	03_run_playbooks.py, playbook_results/
5. Ανάλυση συμπεριφοράς	04_analyze_sessions.py, analysis/, figures/
6. Realism & fingerprinting	04_analyze_sessions.py, figures/fig5

Task	Υλοποιείται από
7. Ηθική & ασφάλεια	README.txt, σχόλια σε 00_config.py /02_ssh_honeypot.py
Προαιρετικό: intent classifier	05_intent_classifier.py, figures/fig6, fig7

Πέραν του κώδικα, η υποβολή περιλαμβάνει την παρούσα αναφορά, το README.txt, το SCHEMA.md, τον κατάλογο εξαρτήσεων (requirements.txt), τον φάκελο llm_prompts/ και τον φάκελο screenshots/.